

Исключение - возможная, предусмотренная ситуация в выполнении программы.

Причины ошибок

1. Действия программиста
(утечки памяти)
2. Действия пользователя
(некорректный ввод)
3. Ошибка времени выполнения
(подключение к бд)

Способы обработки

1. if... else...

Используем коды возврата ошибок.

Например, передаем параметр, куда запишется результат по ссылке, а функция возвращает код ошибки или bool:

```
bool foo(double x, double& y)
{
    // все плохо
    if (abs(x) == 0) return false;
    y = 1 / x;

    // все хорошо
    return true;
}
```

Еще можно создать Enum со статусом выполнения:

```
enum class Status
{
    OK, DIVZERO, NEGATIVE
}

int foo(double x, double& y)
{
    if (x < 0) return Status::NEGATIVE;
    if (x == 0) return Status::DEVZERO;

    y = 1 / x;

    return Status::OK;
}
```

Затем через switch-case обрабатываем результат.

Для таких штук в C++ придумали отдельный тип данных

2. try, catch, throw

```
try {
    // тут может быть ошибка
} catch (exception_type exception_1) {
    // тут мы обрабатываем exception_1
} catch (exception_type exception_2) {
    // тут мы обрабатываем exception_2
} catch (...) {
    // тут мы ловим все остальные ошибки
}
```

Если в `try` возникает ошибка, то происходит передача управления в один из `catch` ей сверху вниз.

```
int a, b;
try {
    std::cin.exceptions(std::istream::failbit);
    std::cin >> a >> b;
    std::cout << a / b << std::endl;
} catch (...) {
    std::cout << "Error!" << std::endl;
}
```

В коде выше `catch` ловит все ошибки, но если попытаться поделить на 0, то никакого "Error" выведено не будет. C++ считает это аппаратной ошибкой и в `catch` управление не передаст.

Не все ошибки C++ кидает сам. Часть из них он игнорирует и их нужно вызывать самостоятельно через `throw`:

```
int a, b;
try {
    std::cin.exceptions(std::istream::failbit);
    std::cin >> a >> b;

    if (b == 0) throw 1;
    if (y == 42) throw 4.2;

    std::cout << a / b << std::endl;
}
catch (int code) {
    if (code == 1) {
        std::cout << "Devision by zero!" <<
std::endl;
    }
}
```

```

catch (double code) {
    std::cout << "Рандомный эксепшн" << std::endl;
}
catch (const ios_base::failure& e) {
    std::cout << e.what() << std::endl;
}
catch (...) {
    std::cout << "Unkown exception!" << std::endl;
}

```

`catch` ориентируется на тип данных и... все. Внутри надо писать `if... else...` или `switch-case` (пункт 1).

Нужно подключить `<std::except>` для нормальных типов ошибок типо `std::runtime_error`.

Цепочки `catch` ей нужно строить грамотно, т.к. одни ошибки входят в другие (`std::exception` -> `std::logic_error` -> `std::invalid_argument`).

Можно писать свои классы ошибок:

```

class DivisionByZero : public exception
{
    double x;

public:
    // конструктор для throw
    DivisionByZero(double x) : x{x} {}

    // переопределяем метод под наши нужды
    const char* what() const noexcept override
    {
        return "Division by zero";
    }
}

```

```
};  
  
if (x == 0) {  
    // вызываем  
    throw DivisionByZero(x);  
}
```

Плюсы этого подхода

1. Разделение логики программы от обработки исключений
2. Возможность самостоятельно генерировать исключения
3. Можно получить объекты с подробным описанием ошибок
4. Единый стиль во всех языках программирования